

PRÁCTICA 3: COMANDOS SOBRE PROCESOS

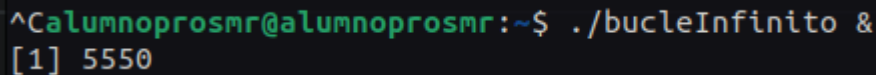
Para realizar esta práctica debes crear el script *bucleinfinito* que se va a ejecutar de manera indefinida. Un script es un fichero cuyas líneas son comandos que se van a ejecutar. Por tanto, nos deberemos asegurar de que ese fichero tenga permiso de ejecución ya que tendremos que ejecutar su contenido.

El script *bucleInfinito* contiene lo siguiente:

```
while [ 1 -lt 10 ]; do  
    echo > /dev/null  
done
```

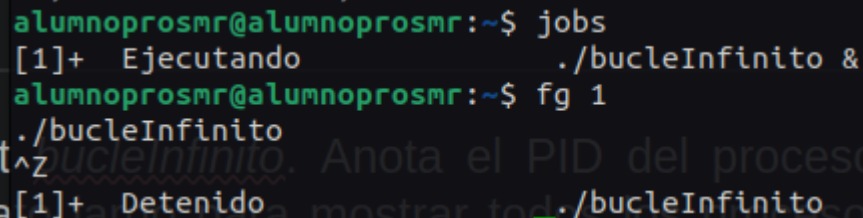
1. Ejecuta el script *bucleInfinito* ¿qué ocurre? ¿Cómo paramos la ejecución de este script?

Que se ejecuta el script y se queda en ejecución



```
^Calumnoprosmr@alumnoprosmr:~$ ./bucleInfinito &  
[1] 5550
```

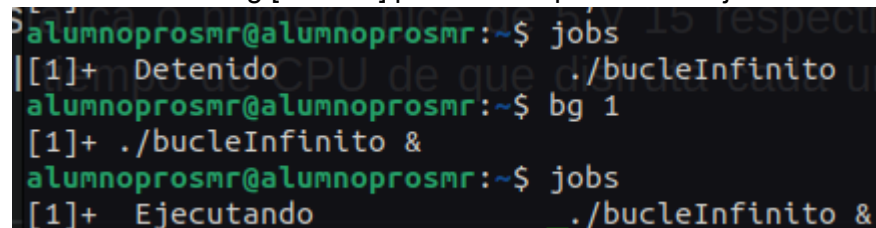
Con jobs vemos el estado del proceso, en este caso está ejecutándose y con el comando fg [número] ponemos el proceso en primer plano, a continuación pulsamos Ctrl+Z para detenerlo.



```
alumnoprosmr@alumnoprosmr:~$ jobs  
[1]+  Ejecutando          ./bucleInfinito &  
alumnoprosmr@alumnoprosmr:~$ fg 1  
./bucleInfinito  
^Z  
[1]+  Detenido             ./bucleInfinito
```

2. Ejecuta en segundo plano el script *bucleInfinito*. Anota el PID del proceso creado. Ahora ejecuta el comando ps en formato largo para mostrar todos los procesos que se estén ejecutando en tu terminal y analiza la información que obtengas.

Con el comando bg [número] ponemos el proceso en ejecución en segundo plano



```
alumnoprosmr@alumnoprosmr:~$ jobs  
[1]+  Detenido             ./bucleInfinito  
alumnoprosmr@alumnoprosmr:~$ bg 1  
[1]+  ./bucleInfinito &  
alumnoprosmr@alumnoprosmr:~$ jobs  
[1]+  Ejecutando          ./bucleInfinito &
```

Con ps la mostramos los procesos en ejecución

```
alumnoprosmr@alumnoprosmr:~$ ps -la
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY      TIME CMD
4 S   0       1585     1583  1   80   0  - 1624744 -   tty2     00:00:43 Xorg
0 S  1000      1601     1583  0   80   0  - 56459 do_pol tty2     00:00:00 gnome-session-b
1 R  1000      6266     5245 86   80   0  - 3549 -   pts/0    00:02:03 bash
4 R  1000      6275     5245  0   80   0  - 3844 -   pts/0    00:00:00 ps
```

3. Vuelve a lanzar en segundo plano el proceso dos veces más, también en segundo plano, pero ahora con una prioridad estática o número nice de 5 y 15 respectivamente. Observa qué influencia tiene esto en el tiempo de CPU de que disfruta cada uno de los procesos.

```
alumnoprosmr@alumnoprosmr:~$ nice -5 ./bucleInfinito &
[2] 6371
alumnoprosmr@alumnoprosmr:~$ nice -15 ./bucleInfinito &
[3] 6373
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando                ./bucleInfinito &
[2]- Ejecutando                nice -5 ./bucleInfinito &
[3]+ Ejecutando                nice -15 ./bucleInfinito &

alumnoprosmr@alumnoprosmr:~$ ps e
  PID TTY      STAT   TIME COMMAND
 1583 tty2     Ssl+   0:00 /usr/libexec/gdm-x-session --run-script env GNOME_SHELL_SESSION_MODE=ubu
 1601 tty2     Sl+    0:00 /usr/libexec/gnome-session-binary --session=ubuntu USER=alumnoprosmr XDG
 5245 pts/0    Ss     0:00 bash SYSTEMD_EXEC_PID=1739 SSH_AUTH_SOCK=/run/user/1000/keyring/ssh SESS
 6266 pts/0    R      9:45 bash SYSTEMD_EXEC_PID=1739 SSH_AUTH_SOCK=/run/user/1000/keyring/ssh SESS
 6371 pts/0    RN     2:13 /bin/sh ./bucleInfinito SHELL=/bin/bash SESSION_MANAGER=local/alumnopros
 6373 pts/0    RN     2:09 /bin/sh ./bucleInfinito SHELL=/bin/bash SESSION_MANAGER=local/alumnopros
 6477 pts/0    R+     0:00 ps e SHELL=/bin/bash SESSION_MANAGER=local/alumnoprosmr:@/tmp/.ICE-unix/
```

4. Cambia el número nice del proceso que tiene prioridad estática 15 para ponérsela a 5. ¿Te deja? ¿Por qué? Cambia el número nice del proceso que tiene prioridad estática 5 para ponérsela a 15. ¿Te deja? ¿Por qué? Comprueba qué efectos tiene esto en el reparto de la CPU.

Que no deja cambiarle la prioridad a más pero si a menos

5. Detén el proceso al que le has cambiado la prioridad en el ejercicio anterior mandándole la señal correspondiente. Comprueba que se ha parado haciendo uso del comando del comando jobs.

```
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando          ./bucleInfinito &
[2]- Ejecutando          nice -5 ./bucleInfinito &
[3]+ Ejecutando          nice -15 ./bucleInfinito &
alumnoprosmr@alumnoprosmr:~$ fg 2
nice -5 ./bucleInfinito así ha sido con el comando jobs.
^Z
[2]+ Detenido            nice -5 ./bucleInfinito
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando          ./bucleInfinito &
[2]+ Detenido            nice -5 ./bucleInfinito
[3]- Ejecutando          nice -15 ./bucleInfinito &
alumnoprosmr@alumnoprosmr:~$ fg 3
nice -15 ./bucleInfinito
^Z
[3]+ Detenido            nice -15 ./bucleInfinito
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando          ./bucleInfinito &
[2]- Detenido            nice -5 ./bucleInfinito
[3]+ Detenido            nice -15 ./bucleInfinito
```

6. Reanuda ahora dicho proceso mandándole la señal correspondiente y vuelve a comprobar que así ha sido con el comando jobs.

```
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando      ./bucleInfinito &
[2]-  Detenido       nice -5 ./bucleInfinito
[3]+  Detenido       nice -15 ./bucleInfinito
alumnoprosmr@alumnoprosmr:~$ bg 2
[2]-  nice -5 ./bucleInfinito &
alumnoprosmr@alumnoprosmr:~$ bg 3
[3]+  nice -15 ./bucleInfinito &
alumnoprosmr@alumnoprosmr:~$ jobws
jobws: orden no encontrada
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando      ./bucleInfinito &
[2]-  Ejecutando     nice -5 ./bucleInfinito &
[3]+  Ejecutando     nice -15 ./bucleInfinito &
alumnoprosmr@alumnoprosmr:~$
```

7. Mata ahora el proceso mandándole la señal correspondiente.

```
alumnoprosmr@alumnoprosmr:~$ jobs
[1]  Ejecutando      ./bucleInfinito &
[2]-  Ejecutando     nice -5 ./bucleInfinito &
[3]+  Ejecutando     nice -15 ./bucleInfinito &
alumnoprosmr@alumnoprosmr:~$ bg 1
bash: bg: el trabajo 1 ya está en segundo plano
alumnoprosmr@alumnoprosmr:~$ fg 1
./bucleInfinito
^C
alumnoprosmr@alumnoprosmr:~$ fg 2
nice -5 ./bucleInfinito
^C
alumnoprosmr@alumnoprosmr:~$ fg 3
nice -15 ./bucleInfinito
^C
alumnoprosmr@alumnoprosmr:~$ jobs
alumnoprosmr@alumnoprosmr:~$
```